
ESP32 Bit Pirate — Complete Hardware Guide

Firmware: ESP32 Bit Pirate · **Upstream:** [geo-tp/ESP32-Bit-Pirate](#) (MIT) **Chip:** ESP32-S3 only (8 MB+ flash) · **Cyber Controller profile:** [esp32-bit-pirate](#) (esptool backend, merged single-bin @ 0x0) **This guide:** purchase → build → flash → integrate into Cyber Controller → use → troubleshoot.

1. Overview

ESP32 Bit Pirate is an open-source firmware that turns an ESP32-S3 board into a "Bus Pirate"-style hardware-hacking multi-tool. From one device you can sniff, send, script, and interactively talk to a wide range of **wired bus protocols** — I2C, SPI, UART / half-duplex UART, 1-Wire, 2-Wire/3-Wire, DIO (digital I/O, PWM, servo), CAN, JTAG/SWD, and I2S — plus **radio protocols** (BLE, Wi-Fi, Sub-GHz, RFID, RF24, Infrared, FM, cellular) on boards that carry the matching hardware. You drive it from a serial CLI, a browser-based Web Serial / Wi-Fi web CLI, or — on the M5 Cardputer — a standalone on-device keyboard/screen interface. It supports Bus-Pirate-style bytecode instructions and Python scripting for automation.

Naming, to avoid confusion: the original **Bus Pirate** is a separate, well-known dedicated hardware tool from Dangerous Prototypes. This is an independent ESP32 *reimagining* by geo-tp. The project was **renamed from "ESP32 Bus Pirate" to "ESP32 Bit Pirate"** during development — they are the same firmware; older articles and the geo-tp.github.io/ESP32-Bus-Pirate site refer to it under the old name.

Cyber Controller flashes this firmware (merged single binary at offset `0x0`) and gives you the **Devices** tab serial CLI to drive its command set — choosing a mode, scanning a bus, and dumping a target chip.

2. Legal & Safety

Authorized hardware research only — on devices you own or are explicitly authorized to test.

Upstream states it is "provided for educational, diagnostic, and interoperability testing purposes only. Do not use it to interfere with, probe, or manipulate devices without proper authorization."

- **Radio modes are regulated.** Wi-Fi deauth, Sub-GHz record/replay, RFID cloning, FM broadcast, infrared and cellular features can break local law (US: CFAA/FCC; EU and others similar) when used against gear or spectrum you don't own. Avoid any unauthorized RF transmission.
- **Electrical safety — this is the big one for a bus tool.** Upstream warning: *"Devices should only operate at 3.3 V or 5 V. Do not connect peripherals using other voltage levels — doing so may damage your ESP32."* The ESP32-S3's **GPIO are 3.3 V logic and are not 5 V tolerant** — driving a 5 V signal line straight into a GPIO can destroy the pin or the chip. Use a level shifter for 5 V signal lines, and always share a common ground with the target.
- The authors disclaim responsibility for misuse. Keep work on a bench, on your own boards/EEPROMs.

3. Hardware & Purchasing

Bit Pirate is **ESP32-S3 exclusive** and needs **at least 8 MB of flash** (the merged build is the 16 MB- class image). Pick a board by how much I/O and which radios you want — every board exposes a different GPIO count and a different set of onboard modules.

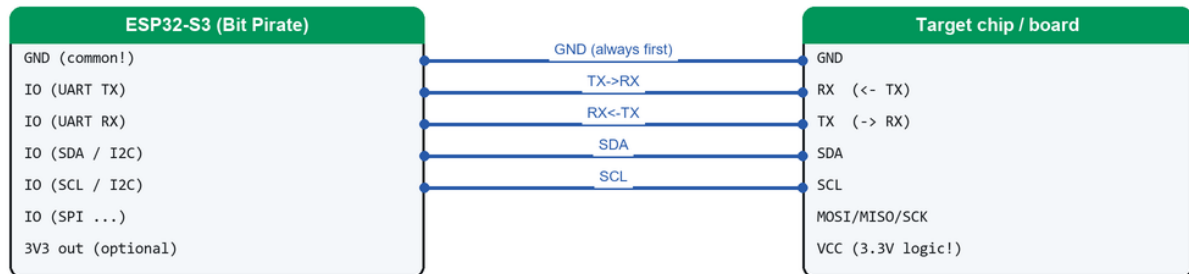
Tier	Board (release asset)	Why / what it carries	Where to buy (search terms)
Most GPIO, cheapest	ESP32-S3 DevKit (s3devkit, or s3devkitn16r8 for N16R8 16 MB)	20+ free GPIO, 1 button — best for raw bus wiring	AliExpress/Amazon: "ESP32-S3-DevKitC-1 N16R8"
Pocket all-in-one	M5 Cardputer (cardputer)	Screen, keyboard , mic/speaker, IR TX, SD, battery — the only standalone board; 2 Grove GPIO	M5Stack store / Amazon: "M5 Cardputer"
Tiny, exposed pins	Seeed XIAO ESP32-S3 (xiaos3)	9 exposed GPIO, 1 button — small bench probe	Seeed Studio / Amazon: "Seeed XIAO ESP32-S3"
Radio-loaded	LilyGo T-Embed CC1101 (tembedcc1101)	Adds CC1101 Sub-GHz, PN532 RFID, IR TX/RX, screen, encoder, SD, battery	LilyGo store / AliExpress: "LilyGo T-Embed CC1101"
Radio + RF24	LilyGo T-Embed CC1101 Plus (tembedcc1101plus)	T-Embed CC1101 features plus NRF24	LilyGo store: "T-Embed CC1101 Plus"
Screen, no radio add-ons	LilyGo T-Display-S3 (tdisplays3) / T-Embed (tembeds3)	Display + a handful of GPIO	LilyGo store: "LilyGo T-Display-S3" / "T-Embed"
Stick form factor	M5 StickC Plus2 / Stick S3 (sticks3)	Screen, IR TX/RX, IMU, buttons, battery, ~13 GPIO	M5Stack store: "M5StickC Plus2" — <i>verify it is the S3 variant</i>
Bare minimal	M5 StampS3 (stamps3) / M5 AtomS3 Lite (atoms3lite)	Compact, few exposed GPIO, 1 button (Atom adds IR TX)	M5Stack store: "M5 StampS3" / "M5 AtomS3 Lite"
Newer Cardputer	M5 Cardputer ADV (cardputeradv)	More GPIO (Grove + header), IMU, keyboard/screen	M5Stack store: "M5 Cardputer ADV" — <i>newer; verify availability</i>

The asset filenames above are the actual binaries from the upstream release (latest tag at time of writing: **v1.6**), named `bit_pirate_16_<board>.bin`. Any ESP32-S3 board with 8 MB+ flash can in principle run it, but **the default firmware pin mapping may not match a board it wasn't built for** — prefer one of the listed targets.

Accessories: a real **USB-C data cable** (not charge-only); **jumper wires / Dupont leads** and a **breadboard** for bus wiring; a **logic-level shifter** for any 5 V signal lines; optional **microSD** (FAT32) on boards that have a slot for capture/dump storage; **SOIC-8 test clip / SPI flash clip** for in-circuit EEPROM/flash dumping. Two optional companion projects exist (verify before buying): the **ESP32 Bus Expander** (adds 5 GHz Wi-Fi / extra radios over GPIO) and the **ESP32 Bit Pirate Dock** (lets an S3 DevKit use original Bus Pirate adapters/accessories).

4. Building / Assembly

Wiring — ESP32 Bit Pirate to a target device (probe)



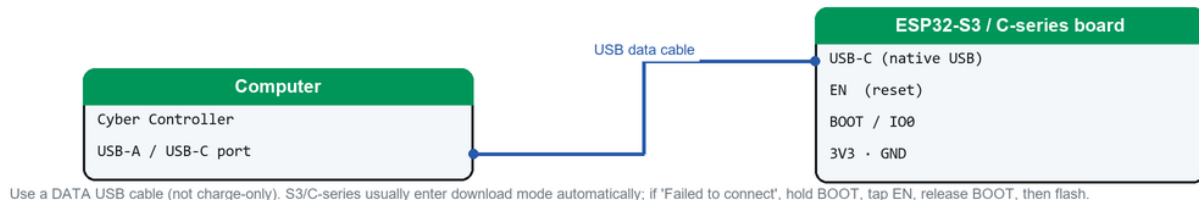
ALWAYS tie GND first. ESP32 GPIO are 3.3V — do NOT connect 5V logic without a level shifter. Pick the bus (UART/I2C/SPI) in the Bit Pirate menu; only wire the lines that bus uses.

Pin layout & connections — verify exact GPIO against the upstream schematic and the table below.

- **Pre-built boards (Cardputer, T-Embed, XIAO, StampS3, Atom, Stick):** no assembly — just a data cable. Onboard modules (CC1101, PN532, NRF24, IR) are already wired on the boards that include them.
- **DevKit bench rig (DIY):** the "build" here is **wiring to the target chip**, not soldering the ESP32. Connect the target's bus pins to the ESP32-S3 GPIO that the firmware assigns for that mode, and **always tie grounds together**. Typical per-protocol wiring:
- **I2C:** SDA↔SDA, SCL↔SCL, GND↔GND, plus pull-up resistors (often ~4.7 kΩ to 3.3 V) if the target lacks them.
- **SPI:** MOSI, MISO, SCK, CS each pin-to-pin, GND common (great for dumping 25-series SPI flash / EEPROM).
- **UART: cross over** TX↔RX and RX↔TX, GND common (Bit Pirate can auto-detect baud rate).
- **1-Wire:** single DATA line + GND, with a pull-up to 3.3 V (iButton / 1-Wire EEPROM).
- **JTAG/SWD, CAN, 2-/3-Wire:** wire the standard pins for that bus to the firmware's assigned GPIO.
- **Power:** you can power a small target from the board's 3.3 V/5 V rail, but keep **signal logic at 3.3 V** (level-shift 5 V signals). Never feed >5 V anywhere on the ESP32.
- **Exact GPIO numbers differ per board.** *Verify: the per-mode default pin assignments on the upstream wiki page "99-Devices Pinout" before wiring — do not assume a pin.*
- **Drivers:** most ESP32-S3 boards enumerate over **native USB (USB-CDC/JTAG)** and need no driver. Some carry a USB-serial bridge instead — *verify your board's chip (e.g. CH9102 → CH34x driver, or CP210x).*

5. Flashing & First Run (via Cyber Controller)

Flashing connection — ESP32 (native USB)



How to connect the board to flash it via Cyber Controller (native-USB ESP32).

1. Connect the board by USB; open Cyber Controller → **Flash** tab.
2. **Port:** pick the board's COM/tty (click *Refresh* if missing → driver/cable issue).
3. **Firmware Profile:** esp32-bit-pirate.
4. **Board / variant:** choose your exact board (e.g. *M5 Cardputer*, *XIAO ESP32-S3*, *T-Embed CC1101*, *S3 DevKit*). The profile maps every board to chip **esp32s3** and selects the matching `bit_pirate_16_<board>.bin`. Picking the wrong board can leave display/modules/pins misconfigured.
5. Click **Flash**. Cyber Controller uses the **esptool** backend to fetch the latest GitHub release asset for your board and writes the **merged single binary at offset 0x0** (flash mode `qio`, freq `80m`). This is the same "web-flasher class" image the upstream one-click flasher writes.
6. **First run:** on screen boards the Bit Pirate UI appears; on headless boards (DevKit/XIAO/Stamp) open the **Devices** tab serial CLI at **115200 baud** and you'll get the interactive prompt. Type `help`.

Alternative upstream flashing paths (for reference): the official **Web Flasher** at geo-tp.github.io/ESP32-Bit-Pirate (Web Serial, one click) and **M5Burner** (StickS3/AtomS3/StampS3/ Cardputer categories). Inside Cyber Controller you don't need these — the profile handles it.

6. Integrate into Cyber Controller

- **Profile:** `esp32-bit-pirate` — **esptool** backend, chip **esp32s3** (fixed), **merged single .bin written at 0x0**, default baud **115200**, latest release resolved from the upstream GitHub release. This is a single-file image (no separate bootloader/partition offsets, unlike Marauder).
- **No suicide / self-erase:** this profile is **not** suicide-capable (`supports_suicide: false`) — there is no destructive self-wipe command for it. To remove the firmware, use the Flash tab's **Erase Flash**.
- **Control:** open the **Devices** tab → select the port → set firmware to **ESP32 Bit Pirate** → connect at **115200**. You get the interactive Bit Pirate CLI in the persistent terminal.
- **Command flow:** Bit Pirate is **modal** — you first enter a protocol mode, then run that mode's commands. Core verbs include `help` (list commands), `mode` (select a mode such as I2C/SPI/UART/1WIRE/ etc.), `scan` (enumerate devices on the selected bus), and `sniff` (capture bus traffic). Mode-specific tools handle EEPROM/flash dump, glitching, bridging, and so on.
- **Backup first:** use **Backup** in the Flash tab to dump current flash before overwriting any board.
- **Scope of integration:** Cyber Controller's value here is flashing the right per-board binary and giving you a stable, logged serial console; the deep per-mode interaction happens through Bit Pirate's own CLI.

7. Usage (end-to-end)

A typical wired-bus session — say, dumping an I2C EEPROM on your own board: 1. **Wire it up** (§4): SDA, SCL, GND to the firmware's I2C GPIO, with pull-ups; share ground; keep 3.3 V logic. 2. **Connect** in the Devices tab at 115200; type `help` to confirm the prompt is live. 3. **Enter the mode**: select **I2C** (via `mode`/the mode menu). 4. **Scan the bus**: `scan` → Bit Pirate lists responding I2C addresses; confirm your target's address shows. 5. **Interact**: use the I2C mode's tools to read registers / dump the EEPROM (and write, if authorized). 6. **Other buses follow the same pattern**: SPI mode → dump 25-series flash; UART mode → auto-detect baud and bridge/read/write; 1-Wire → read an iButton; DIO → drive PWM/servo or read pin states. 7. **Radio modes** (only on boards with the hardware — e.g. CC1101 on T-Embed CC1101): Sub-GHz analyze/ record/replay, RFID read/clone, BLE/Wi-Fi scan — **authorized targets and lawful spectrum only**. 8. **Automate**: drive the same serial commands with the upstream **Python scripting** examples (e.g. EEPROM dump, GPIO interaction, LED animation) from the "ESP32 Bit Pirate Scripts" repo, or use Bus-Pirate-style bytecode instructions. 9. **Stop / disconnect** to free the port when done.

The browser **Web Serial / Wi-Fi web CLI** and (Cardputer-only) **standalone** interface are upstream alternatives to the serial console — handy when you're not at Cyber Controller, but the same command set.

8. Troubleshooting

- **No COM port:** most S3 boards are native USB — try another **data** cable and USB port first; if the board has a USB-serial bridge, install its driver (*verify chip: CH9102/CH34x or CP210x*). On Linux add your user to `dialout` and `replug`.
- **Flash fails / "Failed to connect":** put the S3 in **download mode** — hold **BOOT** while tapping **RESET** (or hold BOOT while plugging in), then flash; lower the flash baud in Settings; close any other serial monitor (including a browser Web Serial tab) holding the port.
- **Wrong board picked** → **blank screen / dead modules / wrong pins:** re-flash selecting the **exact** board variant so the matching `bit_pirate_16_<board>.bin` and its pin map are used.
- **"Flash too small" / boot-loop:** the build needs **8 MB+** flash — verify the board is genuinely 8/16 MB (clones can mislabel). Use **Erase Flash** then re-flash.
- **I2C/SPI scan finds nothing:** check **common ground**, add **pull-ups** (I2C), confirm you crossed **TX/RX** for UART, confirm **3.3 V** logic (level-shift 5 V lines), and that you're in the right **mode**. Re-check the GPIO assignment against the wiki "99-Devices Pinout".
- **Damaged GPIO after probing:** usually a 5 V signal on a 3.3 V pin — that pin/chip may be dead; this is why level shifting and the 3.3 V/5 V rule matter.
- **Radio mode missing/non-functional:** that board lacks the hardware (e.g. no CC1101/PN532/NRF24) — only the boards listed with those modules support those modes.
- **Verify failed:** re-run flash; a bad cable or counterfeit flash chip is the usual cause.

9. Sources

- Upstream repo: <https://github.com/geo-tp/ESP32-Bit-Pirate> (README, releases, MIT license).
- Upstream wiki: <https://github.com/geo-tp/ESP32-Bit-Pirate/wiki> (Quick Start, Terminal Selection, per-mode pages, "99-Serial", "99-Instructions Syntax", "99-Devices Pinout", build instructions).
- Web flasher / web tools: <https://geo-tp.github.io/ESP32-Bit-Pirate/> (board list, Web Serial flashing).
- Release assets (board binaries, tag v1.6): <https://github.com/geo-tp/ESP32-Bit-Pirate/releases/latest> — files named `bit_pirate_16_<board>.bin`.
- Project background / rename from "ESP32 Bus Pirate": Hackster.io, Hackaday.io, LinuxGizmos coverage.
- Cyber Controller profile: `src/config/profiles/bit_pirate.json` (boards, esp32s3 chip map, merged `0x0` offset, esptool backend, 115200 baud). Note in that file: per-board offset/pin details pending hardware verification (Stage-5 gate) — *verify on real hardware before relying on a specific board's pin map*.
- Verify current board availability, prices, driver chips, and exact GPIO pin maps at use time; vendor links and board revisions change.